

Exploring the use of Long Short Term Memory recurrent neural networks (LSTMs) in estimating Value-at-Risk for backtesting applications in Financial Risk Management

Ryan Kai Yan Seet
Term Project Group 165
ISYE 6740 Project Report

November 24, 2024

Abstract

Value-at-Risk (VaR) is a widely used measure in financial risk management, offering a quantitative metric for potential portfolio losses. Conventional methods, such as historical and Monte Carlo simulations, calculate VaR by forecasting potential price movements and determining the maximum expected loss within a specified confidence interval (e.g., 95%) [1]. Backtesting, which is a key validation method used in the industry, compares forecasted losses from VaR models with actual losses to evaluate their accuracy [2].

In recent years, the advent of deep learning methods such as Long Short Term Memory (LSTM) neural networks provide new opportunities to enhance VaR methodologies. LSTMs capture long-term dependencies and nonlinearities in time series, making them suitable for modeling equity market returns and generating P&Ls.

This paper investigates an LSTM-based methodology for VaR calculation, combining LSTM forecasts with Monte Carlo simulations to produce a rolling LSTM-VaR threshold over a 1.5-year test period. Results suggest that LSTM-VaR seems to generally be more responsive to financial market crisis period-events than the contemporary historical or monte-carlo methods, but they also tend to predict higher losses in the negative direction, possibly indicating that the contemporary methods may be potentially underestimating risk.

1 Introduction

VaR seeks to answer the question, *What is the minimum amount that I can expect to lose with a certain probability over a given horizon?* [1]. There are some issues with these contemporary methods of calculating VaR. For example, financial markets tend to underreact to minor events and overreact to major ones. Past studies also indicate that returns also exhibit some degree of market asymmetry, for example, reacting slightly stronger to positive than negative events [3]. This indicates that while contemporary methods of estimating VaR assume that returns follow a normal distribution, this does not

hold in reality.

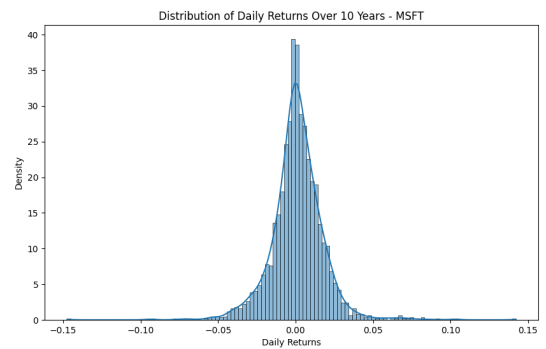


Figure 1: Histogram showing the distribution of a ten-year period of daily historical returns of a sample US equity, with slightly high kurtosis and fatter tails i.e., Microsoft (MSFT US Equity) stock.

This has led to other types of metrics, such as Expected Shortfall (ES), which describe how large losses are on average when they exceed the VaR threshold [1]. VaR, along with ES-related metrics are used in tandem with stress tests [1], which apply simulated parameter changes to financial pricing models based on historical events or potential future events in order to provide a comprehensive approach to risk management.

Recent advances in Deep learning and RNNs through techniques such as Long short-term memory (LSTM) recurrent neural networks [5] could potentially offer a more concise methodology in capturing the non-linear relationships of stock returns more accurately and long-term dependencies in time series data more efficiently, potentially leading to a more reliable risk management methodology.

2 Related Work

Similar work has been conducted in the field such as via multivariate time series forecasting in [4] where multi-layered LSTM models are used on multivariate time series data to generate an entirely new set of time-series data using LSTM models with stacked autoencoders

(i.e., LSTM-SAE). This was found to be useful for generating realistic market-based stress-testing scenarios.

In [6], where the focus is slightly different from risk management, and mainly on trading applications and analyzing portfolio performance, rolling 4-year LSTM based models were used in model training to optimize trading strategies, leveraging return prediction and model optimization.

The approach taken in this report however, is comparatively simple, and involves the use of a single LSTM model slightly complex loss function to be discussed in section 4.7, and will be focused on evaluating the returns of variants of portfolios generated using 15 years of historical return data of stocks present in the S&P 500 index.

3 Data Description

The datasets used for this study consist of adjusted closing prices for S&P 500 equities with a 15-year historical window that spans from November 2009 to November 2024. These data were retrieved using the Yahoo Finance API library in Python, resulting in a total of 435 distinct time series of adjusted close price levels.

In order to simulate actual market-cap weighted performance of the S&P 500 Index, the market capitalizations are also obtained in order to obtain precise historical weights for each historical component. For better analysis on the impact of different industry factors on the changes in rolling VaR, data on Sector, Industry, and Sub-Industry classifications for the equities were scraped and integrated. All data is stored in .csv files which are then accessed using the pandas library.

- Adjusted closing prices from November 25 2009 to November 19 2024. `sp500_adjusted_close_cleaned.csv` and market capitalization data `sp500_market_caps.csv`. Scraped industry and sector data `sp500_stock_info.csv`.
- The data is then split into training and test sets for two scenarios, a base case analysis with a test period consisting of the most recent 1.5 year window and a crisis-period analysis using a test period for the Covid-19 financial crisis.
- Data preprocessing steps such as scaling, normalization and sequence generation are also specified and initialized for the LSTM model inputs.

Principal Components analysis (PCA) is also conducted on these time series price-levels[5] in order to create five additional different portfolios in addition to the initial two equal-weight and market-weighted-based S&P 500 portfolios. A total of five of these principal component (PC) based portfolios were created. The return vectors of all portfolios were subsequently converted into tensors suitable for training in the LSTM models. We use

this method to analyze how LSTM VaR would perform given equity-based portfolios with differing weights in different sectors.

4 Methodology

Details of the various methodologies utilized are listed and expounded upon in the sections below.

- **PCA Portfolio Construction:** Starting with the $T \times N$ returns matrix $\mathbf{R}$ (T time periods, N stocks), we standardize:

$$\mathbf{X} = \frac{\mathbf{R} - \mu}{\sigma} \quad (1)$$

The PCA decomposition gives:

$$\mathbf{X} = \mathbf{U}\Sigma\mathbf{V}^T \quad (2)$$

where:

- $\mathbf{U}$ ($T \times K$) contains the time series of individual portfolio returns
- Σ ($K \times K$) contains the standard deviations of these portfolio returns
- $\mathbf{V}$ ($N \times K$) contains the portfolio weights that generate these returns

4.1 Portfolio Weight Construction

For each principal component k (where $k = 1$ to $5, \dots, K$ and K is the number of components selected), the weights for each equity within the portfolio are computed as:

$$w_{k,i}^{PCA} = \frac{|v_{k,i}|}{\sum_{j=1}^N |v_{k,j}|} \quad (3)$$

where:

- $w_{k,i}^{PCA}$ is the weight of stock i in the k -th PCA portfolio
- $v_{k,i}$ is the i -th element of the k -th principal component loading vector
- The absolute values enforce non-negative weights
- The presence of the divisor normalizes the sum of these weights to 1.

The sector exposure for an industry sector s is then obtained as follows:

$$w_{k,s}^{sect} = \sum_{i \in s} w_{k,i}^{PCA} \quad (4)$$

where $i \in s$ denotes all stocks in sector s .

4.2 Brief analysis of portfolios created

The portfolios created have their calculated correlations, sector weightages and cumulative returns for June 2024 to November 2024 summarized in the figure below.

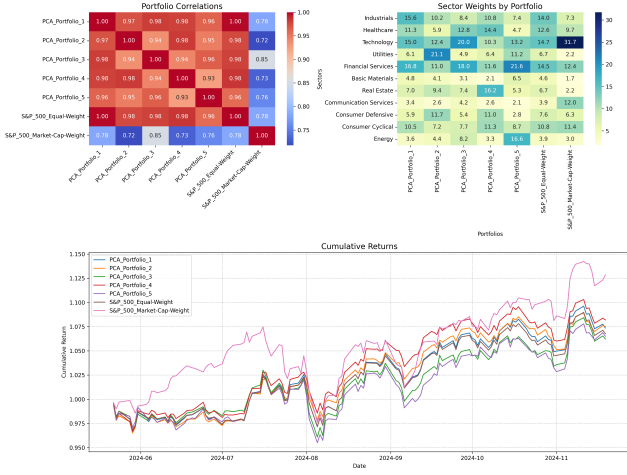


Figure 2: Portfolio correlations, sector weights and a chart showing the latest 6 months of cumulative portfolio returns for all portfolios.

The portfolios have very high correlations with each other since they are obtained from the same universe, i.e. the S&P 500 Index. We see good variations in industry weights, for example, PCA portfolio 2 has a higher concentration in Utilities, while PCA Portfolio 5 has a higher concentration in the Financial Services Sector.

- **Sequence Creation:** We derive a `create_sequences()` function to prepare the LSTM model inputs as follows.

4.3 Details on the input data used

First, the portfolio returns are standardized as follows:

$$\mathbf{r}_{std} = \frac{\mathbf{r} - \mu}{\sigma} \quad (5)$$

Given a time series of portfolio returns:

$$\mathbf{r} = [r_1, r_2, \dots, r_T] \quad (6)$$

where:

- r_t is the return of the portfolio at time t
- T is the total number of observations

4.4 Sequence Formation

For sequence length L and total observations T : 1. A tensor with the following shape is created:

$$\mathbf{X} \in R^{(T-L) \times L \times 1} \quad (7)$$

2. Our target y variable tensor shape is then set as follows:

$$\mathbf{y} \in R^{(T-L) \times 1} \quad (8)$$

Each sequence-target pair (i) is formed using the following equation:

$$\mathbf{X}_i = [r_{std,i}, r_{std,i+1}, \dots, r_{std,i+L-1}] \quad (9)$$

$$y_i = r_{std,i+L} \quad (10)$$

4.5 Final Dataset Structure

The training dataset becomes:

$$\mathcal{D} = \{(\mathbf{X}_1, y_1), (\mathbf{X}_2, y_2), \dots, (\mathbf{X}_{T-L}, y_{T-L})\} \quad (11)$$

where:

- $\mathbf{X}_t \in R^L$ (sequence of L returns)
- $y_t \in R$ (single return value)
- Shape of $\mathbf{X}$ is $(T-L) \times L \times 1$
- Shape of $\mathbf{y}$ is $(T-L) \times 1$

Validation set: $\{\mathbf{X}_{val}, \mathbf{y}_{val}\}$ (last $\min(n/5, 50)$ samples)

The final input and output tensors used with Pytorch are then obtained as follows:

$$\mathbf{X}_{tensor} = \text{torch.tensor}(\mathbf{X}, \text{dtype=float32}) \quad (12)$$

$$\mathbf{y}_{tensor} = \text{torch.tensor}(\mathbf{y}, \text{dtype=float32}) \quad (13)$$

Model Architecture: In the LSTM model, all data prior to the test period is used in horizons (sequence lengths) of 252 days, or 1 year of trading days. A detailed description is explained below. This differs slightly from the methodology discussed in [4], since we are just using a rolling horizon or sequence length of 252 days to predict the next day's (day 253) return value instead of generating an entire time series using autoencoders.

Given portfolio returns $[r_1, r_2, \dots, r_T]$ and sequence length $L = 252$ (one year of trading days): Input sequence examples are as follows:

$$\mathbf{X}_1 = [r_1, r_2, \dots, r_{252}] \rightarrow y_1 = r_{253} \quad (14)$$

$$\mathbf{X}_2 = [r_2, r_3, \dots, r_{253}] \rightarrow y_2 = r_{254} \quad (15)$$

4.6 Batch Size Organization

We then use a typical batch size $B = 32$:

$$\text{Batch}_1 = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \\ \vdots \\ \mathbf{X}_{32} \end{bmatrix} \in R^{32 \times 252 \times 1} \quad (16)$$

This means that a single LSTM training batch will consist of 32 consecutive stacked training sequences of 1 year long sequence lengths, each overlapping by 1 trading day. The number of batches set per epoch (a complete pass through the data set) is then defined as follows:

$$N_{batches} = \left\lceil \frac{T-L}{B} \right\rceil \quad (17)$$

To give a numerical example, for $T = 1000$ days:

- Total sequences: $1000 - 252 = 748$
- Batch size: 32
- Number of batches: $\lceil \frac{748}{32} \rceil = 24$ batches

4.7 Loss Function details

Typically, LSTM models use mean squared errors (MSE) between the predicted variable and the actual values to model a loss function to minimize the discrepancies between the target and predicted values. However, financial returns are known to exhibit high volatility and follow non-normal, unpredictable distributions, so a more complex loss function is proposed in this paper. First, we ensure that volatilities predicted are non-negative and compute standardized returns:

$$\sigma = \max(\sigma, \epsilon) \quad \text{where } \epsilon = 10^{-6} \quad (18)$$

$$z = \frac{y - \mu}{\sigma} \quad (19)$$

A Negative log-likelihood function with 5 degrees of freedom is then implemented as follows. A student's t-distribution is chosen for its ability to model the fatter tails typically seen in equity returns[8]:

$$\mathcal{L}_{NLL} = 0.5(\log(\sigma^2) + 6 \log(1 + \frac{z^2}{5})) \quad (20)$$

To match the distribution shape of actual returns, we then implement skewness and kurtosis matching components within the loss function.

Skewness matching:

$$\mathcal{L}_{skew} = |E[z^3] - (-0.028)| \quad (21)$$

Kurtosis matching:

$$\mathcal{L}_{kurt} = |E[z^4] - 3.578| \quad (22)$$

Since returns also exhibit a high degree of variation (volatility), the following functions are also added so the loss function can control volatility mismatches, as well as mismatches in second degree changes in volatility (variance). Mean volatility loss:

$$\mathcal{L}_{vol_mean} = |\bar{\sigma} - \sigma_{realized}| \quad (23)$$

Volatility variance loss:

$$\mathcal{L}_{vol_var} = |\text{std}(\sigma) - \sigma_{realized}| \quad (24)$$

The combined volatility loss function used is as follows:

$$\mathcal{L}_{vol} = 0.2\mathcal{L}_{vol_mean} + 0.1\mathcal{L}_{vol_var} \quad (25)$$

Lastly, we want to model VaR breach rates as close as possible to the specified confidence interval, i.e. 95%. A breach rate loss component incorporating VaR calculations using a student's t-distribution is implemented as follows. Value at Risk calculation (using a parametric method):

$$VaR = \mu + z_{0.05}\sigma \quad \text{where } z_{0.05} \text{ from t-distribution} \quad (26)$$

Breach rate loss:

$$\mathcal{L}_{breach} = |\text{mean}(1_{y < VaR}) - 0.05| \quad (27)$$

The **final combined loss function** is then obtained as follows, using arbitrary weights:

$$\mathcal{L}_{total} = 0.3\mathcal{L}_{NLL} + 0.2\mathcal{L}_{skew} + 0.3\mathcal{L}_{kurt} + \mathcal{L}_{vol} + \mathcal{L}_{breach} \quad (28)$$

5 Model Training, LSTM VaR computation and Evaluation

The LSTM model function is set up below as follows using a Python class.

$$\text{LSTM}(input_dim = 1, hidden_dim = 100, layers = 2) \quad (29)$$

where the input dimension is the financial time series of prices as defined above, the hidden dimension refers to the model's ability to retain information about past historical returns. A hidden dimension of 100 is used. To speed up the training process, 2 LSTM layers are specified. The output heads are as specified below. The

model predicts the mean and volatility of the return distribution for the following day as per the following equation:

$$\mu = W_{\mu}h_T + b_{\mu} \quad (30)$$

$$\sigma = \text{Softplus}(W_{\sigma}h_T + b_{\sigma}) = \ln(1 + e^{W_{\sigma}h_T + b_{\sigma}}) \quad (31)$$

The mean head predicts the expected return (μ) based on the final hidden state h_T . A Softplus activation function[9], which is a smoother variant of the well-known softmax function is used and ensures that the predicted volatility (σ) outputs are always positive, addressing the inherent non-negativity of volatility.

5.1 Hyperparameters

The hyperparameters specified in the model are as follows:

- Learning rate: $\alpha = 0.001$ initialized small to allow stable convergence.
- Batch size: $B = 32$ As mentioned above, incorporating 32 consecutive sequences.
- Maximum epochs: $E_{max} = 1000$ Maximum number of times the dataset is run though, although the model will likely converge far below this threshold number.
- Early stopping patience: $P = 15$ An early stopping algorithm is implemented, based on reducing validation losses. If validation losses don't reduce after 15 epochs, the model stops training.
- Adam parameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$ An Adam optimizer is used to accelerate gradient descent to improve convergence speeds towards the minima.

5.2 LSTM Model training Loop

The training loop is then initiated as follows. For each epoch e :

1. Training Phase (using the custom loss function specified above):

$$\mathcal{L}_{train}^e = \frac{1}{N_{batch}} \sum_{i=1}^{N_{batch}} \mathcal{L}(f_{\theta}(X_i), y_i) \quad (32)$$

2. Adam Optimizer Update to handle gradient updates with a high degree of stochastic variability (noise) [11]:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (33)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (34)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (35)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (36)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (37)$$

3. Validation Phase (using the custom loss function to compare generated returns with actual returns for the next day):

$$\mathcal{L}_{val}^e = \frac{1}{N_{val}} \sum_{i=1}^{N_{val}} \mathcal{L}(f_{\theta}(X_i^{val}), y_i^{val}) \quad (38)$$

4. Learning Rate Schedule (implemented as follows to achieve smoother convergence in case the model stagnates):

$$\alpha_{e+1} = \begin{cases} 0.5\alpha_e & \text{if no improvement for 5 epochs} \\ \alpha_e & \text{otherwise} \end{cases} \quad (39)$$

5. Gradient Clipping is also implemented to prevent exploding gradients, which is commonly observed in LSTMs [10]:

$$\|\nabla\theta\| \leq 1.0 \quad (40)$$

6. As mentioned above, early stopping based on model validation losses is implemented as follows:

$$\text{Stop if } \mathcal{L}_{val}^e > \mathcal{L}_{val}^{best} \text{ for } P \text{ epochs} \quad (41)$$

5.3 LSTM VaR computations

As described in the introduction to this section, For each time step t , LSTM outputs:

$$\mu_t = \text{predicted mean return} \quad (42)$$

$$\sigma_t = \text{predicted volatility with softplus function} \quad (43)$$

We then utilize these outputs to generate $N = 1000$ P&L scenarios for each prediction using normal distribution-based Monte-Carlo simulations as per the following:

$$r_{t,i} \sim \mathcal{N}(\mu_t, \sigma_t^2), \quad i = 1, \dots, N \quad (44)$$

The generated returns are then transformed back to their original scales to form a numerical array of N P&Ls for the next day's P&L:

$$R_{t,i} = \text{scaler.inverse.transform}(r_{t,i}) \quad (45)$$

For a chosen confidence level of $\alpha = 0.95$, VaR is simply calculated using the following percentile function:

$$\text{VaR}_t = \text{percentile}\{R_{t,i}\}_{i=1}^N, (1 - \alpha) \times 100 \quad (46)$$

This returns a decimal based negative loss value, that represents the 95% confidence interval 1 year horizon daily LSTM VaR predicted by the model. This process is then repeated for the entire test set, i.e., the most recent 1.5 years to obtain rolling LSTM-VaR values that can be used for backtesting.

5.4 LSTM Expected Shortfall (ES) computations

For scenarios beyond the calculated VaR threshold, similar to other contemporary VaR methods, ES is calculated as follows:

$$\text{ES}_t = E[R_{t,i} | R_{t,i} \leq \text{VaR}_t] = \frac{1}{|\mathcal{L}|} \sum_{i \in \mathcal{L}} R_{t,i} \quad (47)$$

where $\mathcal{L} = \{i : R_{t,i} \leq \text{VaR}_t\}$ where:

- $L = \{i : R_{t,i} \leq \text{VaR}_t\}$: Set of scenarios exceeding VaR
- $|\mathcal{L}|$: Number of scenarios in set $\mathcal{L}$
- $R_{t,i}$: Return for scenario i at time t

5.5 Breach Detection

For the latest 1.5 years of actual return data, we then compare the actual portfolio returns of each of the simulated portfolios r_t^{actual} to the rolling LSTM VaR and ES values to confirm whether VaR has been breached:

$$\text{Breach}_t = \begin{cases} 1 & \text{if } r_t^{actual} < \text{VaR}_t \\ 0 & \text{otherwise} \end{cases} \quad (48)$$

3. Calculate risk measures:

$$\text{VaR}_t = \text{percentile}(\mathbf{R}_t, 5) \quad (49)$$

$$\text{ES}_t = E[\mathbf{R}_t | \mathbf{R}_t \leq \text{VaR}_t] \quad (50)$$

5.6 Metrics used for evaluation

The following metrics are used in our final evaluation, and are briefly discussed as follows:

- Breach rate analysis for rolling VaR and ES (historical backtesting approach to see the rate of actual portfolio losses breaching VaR).
- Statistical measures such as evaluating the characteristics of the predicted return distributions compared with that of actual returns. (Q-Q plots to test for normality)
- Standard deviations of LSTM predicted returns vs. actual returns.
- Stress Period performance incorporating a combination of the above metrics, using a specific Covid-19 stress horizon defined as starting from 18th February 2020 to 20th March 2020.

6 Non-LSTM based contemporary VaR calculation techniques

Non-LSTM related methodologies used for computing historical and Monte-Carlo rolling VaRs for comparison purposes follow contemporary methodologies and are specified as follows. We use the full revaluation method in all cases, and focus of 1 year daily VaRs at the 95% confidence interval.

6.1 Historical VaR

[1] Given a rolling window of 1 year, i.e. $W = 252$ trading days: 1. For each day t , take the portfolio return window:

$$\mathbf{R}_t = \{r_{t-W+1}, \dots, r_t\} \quad (51)$$

where R_t represents a vector of portfolio returns over a rolling window of 252 days. 2. Calculate portfolio VaR at confidence level $\alpha = 0.95$:

$$\text{VaR}_t = \text{percentile}(\mathbf{R}_t, (1 - \alpha) \times 100) \quad (52)$$

3. Calculate ES for losses beyond VaR as follows:

$$\text{ES}_t = E[r_i | r_i \leq \text{VaR}_t, r_i \in \mathbf{R}_t] \quad (53)$$

6.2 Monte Carlo VaR

[1] For portfolio weights $\mathbf{w}$:

1. Calculate historical parameters from a recent rolling window of returns:

$$\boldsymbol{\mu} = E[\mathbf{R}_{\text{recent}}] \quad (54)$$

$$\boldsymbol{\Sigma} = \text{Cov}(\mathbf{R}_{\text{recent}}) \quad (55)$$

where $\boldsymbol{\mu}$ is mean returns vector and $\boldsymbol{\Sigma}$ is covariance matrix.

2. Generate scenarios using a normal distribution and random assumptions, along with a computed covariance matrix of returns:

$$\mathbf{S} = \{\mathbf{s}_i\}_{i=1}^N \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad (56)$$

3. Calculate portfolio returns for each scenario:

$$r_p^i = \mathbf{w}^T \mathbf{s}_i, \quad i = 1, \dots, N \quad (57)$$

4. Compute risk measures as follows:

$$\text{VaR} = \text{percentile}(\{r_p^i\}_{i=1}^N, (1 - \alpha) \times 100) \quad (58)$$

$$\text{ES} = E[r_p^i | r_p^i \leq \text{VaR}] \quad (59)$$

where $N = 1000$ scenarios, similar to LSTM-VaR outputs.

7 Analysis of findings and results

7.1 Distribution of predicted vs. actual returns

Looking at the distribution plots of each portfolio's LSTM model-predicted and actual returns for the past 1.5 years (378 trading days), we can make a few interesting observations. Plots for all seven portfolios are displayed in the figures below:

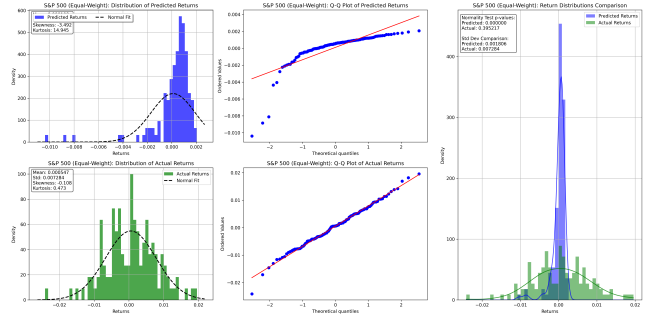


Figure 3: S&P 500 (Equal-Weight) distributions

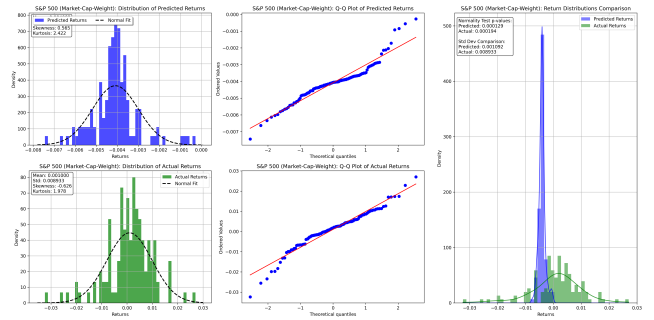


Figure 4: S&P 500 (Market-Cap-Weight) distributions

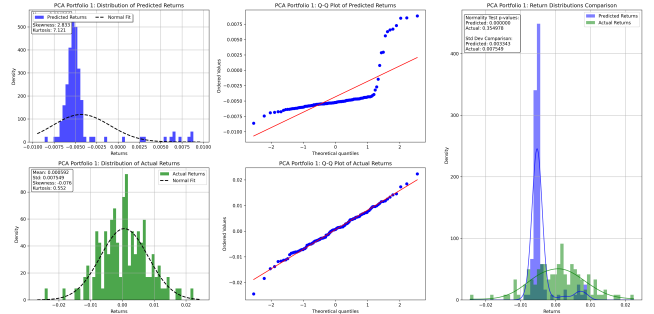


Figure 5: PCA Portfolio 1 distributions (Financial Services, Industrials, Technology heavy)

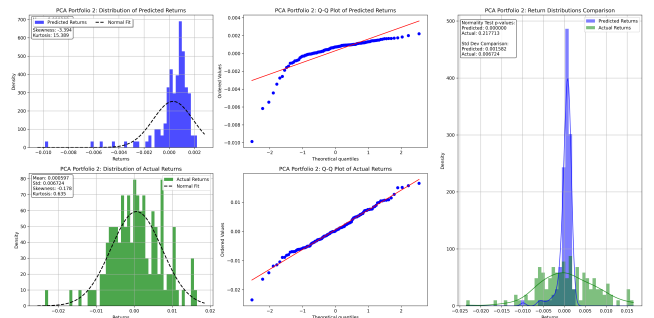


Figure 6: PCA Portfolio 2 distributions (Utilities Heavy)

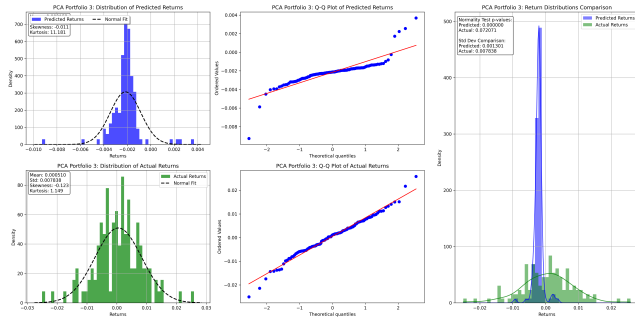


Figure 7: PCA Portfolio 3 distributions (Technology, Financial Services Heavy)

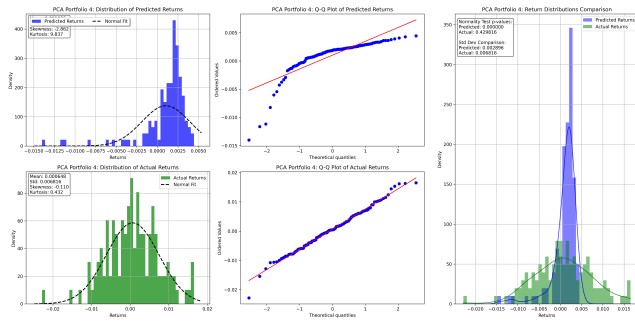


Figure 8: PCA Portfolio 4 distributions (Real Estate Heavy)

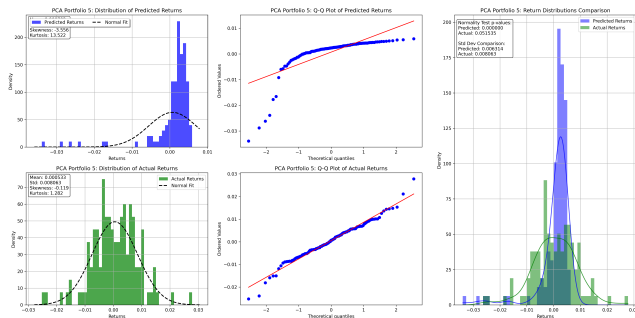


Figure 9: PCA Portfolio 5 distributions (Financial Services, Energy Heavy).png

In general, the predicted returns seem to be more leptokurtic in nature, show significant differences from the actual returns, displaying higher levels of kurtosis and skewness. The actual returns, while not completely normal in most cases seem to be more in line with a normal curve.

Some portfolios such as the Equal weight S&P 500 portfolio, PCA 1 portfolio, PCA 2 portfolio, PCA 4 portfolio and PCA 5 portfolio display significant deviations from normality as per their Q-Q plots for the predicted returns. While there are extreme tail events in actual returns for all portfolios, they diverge to a lesser extent from normality. Some predicted portfolio returns, such as those for the PCA 1 portfolio, PCA 2, PCA 4, and PCA 5 portfolio seem to be evidently asymmetric with left or right skews. These observations potentially indicate that our LSTM model seems to have a tendency to heavily overestimate tail risks as compared to the actual observed portfolio returns.

7.2 VaR backtesting

Historical backtesting of each portfolio's returns is carried out with the calculated rolling VaR and ES values utilizing all three methodologies for a six month period spanning June to November 2024. Detailed results are shown in the plots below. The VaR and ES values calculated are the 1 year daily values at a 95% confidence level.

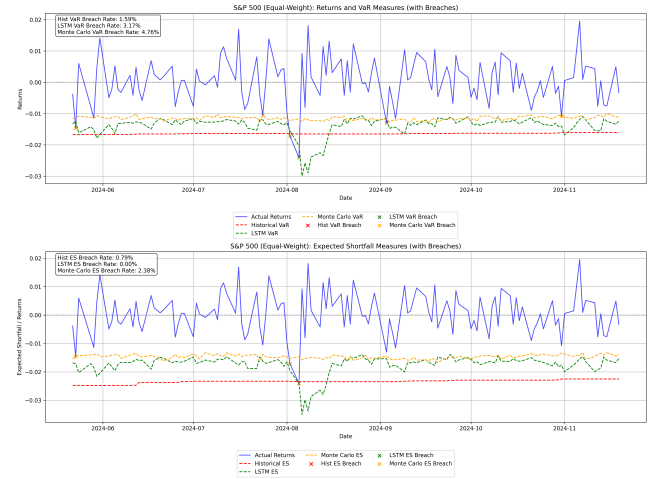


Figure 10: S&P 500 (Equal-Weight) risk comparisons with VaR breaches)

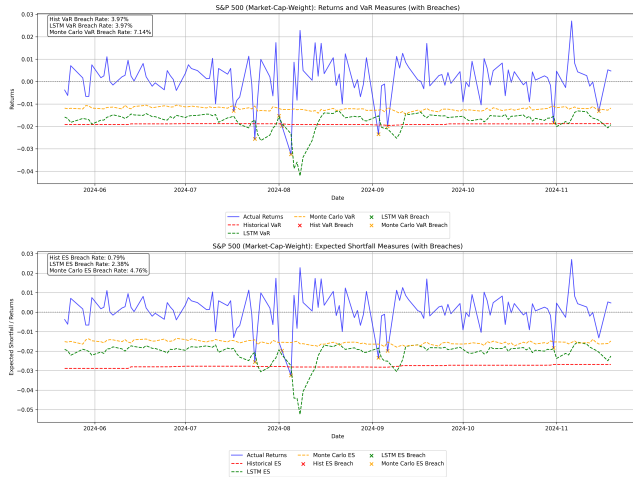


Figure 11: S&P 500 (Market-Cap-Weight) risk comparisons with VaR breaches



Figure 14: PCA Portfolio 3 risk comparisons (Technology, Financial Services Heavy) with VaR breaches

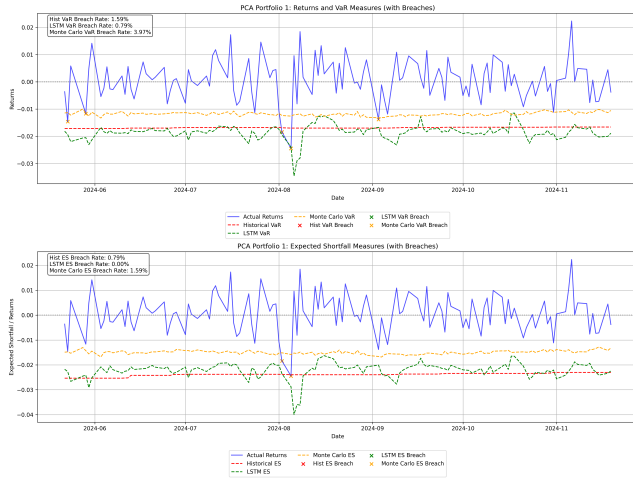


Figure 12: PCA Portfolio 1 risk comparisons (Financial Services, Industrials, Technology heavy) with VaR breaches



Figure 15: PCA Portfolio 4 risk comparisons (Real Estate Heavy) with VaR breaches



Figure 13: PCA Portfolio 2 risk comparisons (Utilities Heavy) with VaR breaches

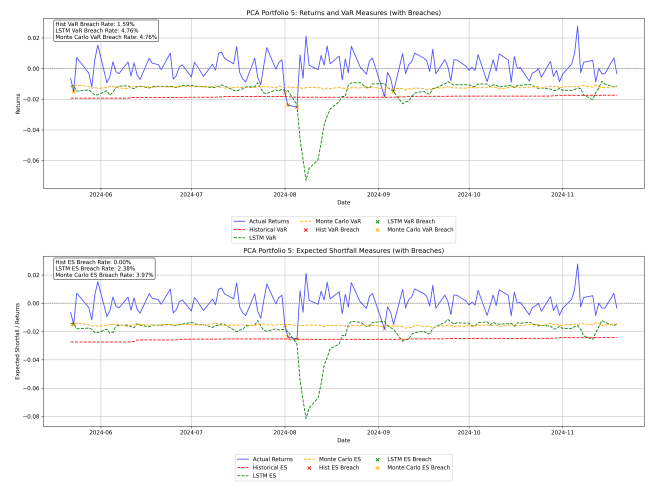


Figure 16: PCA Portfolio 5 risk comparisons (Financial Services, Energy Heavy) with VaR breaches

We notice that for all portfolios, Monte Carlo VaR seems to consistently present the lowest maximum loss thresholds. A very interesting observation here however, is that rolling LSTM-VaR seems to be far more volatile than its contemporaries, being able to capture significant changes in the market, such as a slight spike in the VIX Index captured on August 9th 2024. The VaR value predicted by the LSTM model increases significantly in magnitude before returning back to normal, indicating a strong correlation with actual market volatility. On the other hand, historical and Monte Carlo VaR seem ignorant of sudden market volatility spikes or crisis events, remaining at relatively stable levels.

This behavior is likely due to the nature of the VaR calculation methods. For both historical and Monte Carlo VaR, a long time horizon of one year (i.e., 252 trading days) is used as the basis for simulations, which introduces a smoothing effect. It becomes evident that these traditional VaR estimation methods are less responsive to sudden crisis events and, as a result, may underestimate the associated risks.

In terms of VaR breach rates, it seems that LSTM VaR and Monte-Carlo VaR seem to have higher VaR breach rates than historical VaR, indicating that they are to a certain degree more conservative. However, we also note that in periods of stress, since LSTM-VaR is more responsive to volatility shocks, the larger magnitude of expected loss thresholds computed counterintuitively indicates that there may be 'more room for losses', which would not make much sense when we are looking at backtesting our portfolios. Nonetheless, there is strong evidence that LSTM-VaR provides a more accurate estimation of the non-linearities in returns, especially in capturing sudden spikes in market volatility. In the next section, we will look at how LSTM-VaR changes in a more widely known period of historical market stress.

7.2.1 Evaluating changes in rolling VaR values during the COVID-19 Financial Market Shock

We look into a more specific scenario, using the time horizon of February 18 2020 to March 20 2020, which is the period of the largest dip in the S&P 500 Index, and a common stress horizon utilized by industry researchers[12]. The LSTM model is trained on all available data prior to this period using the specifications above, and the rolling VaRs and breach rates are again plotted as follows.

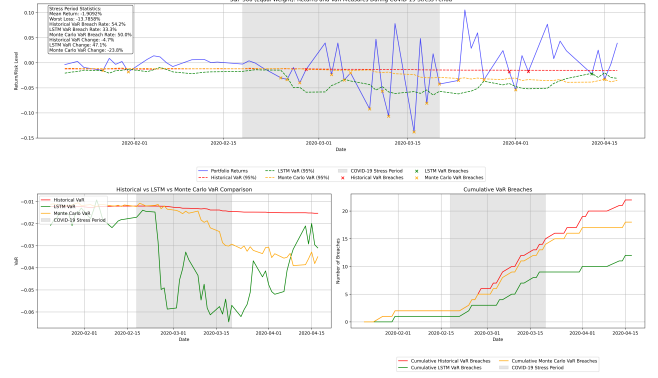


Figure 17: S&P 500 (Equal-Weight) with COVID-19 stress test analysis

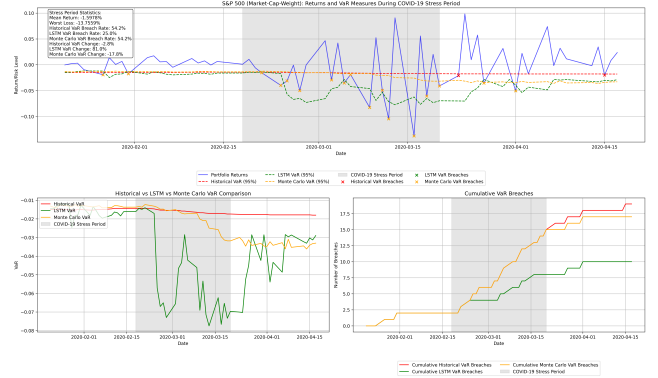


Figure 18: S&P 500 (Market-Cap-Weight) with COVID-19 stress test analysis

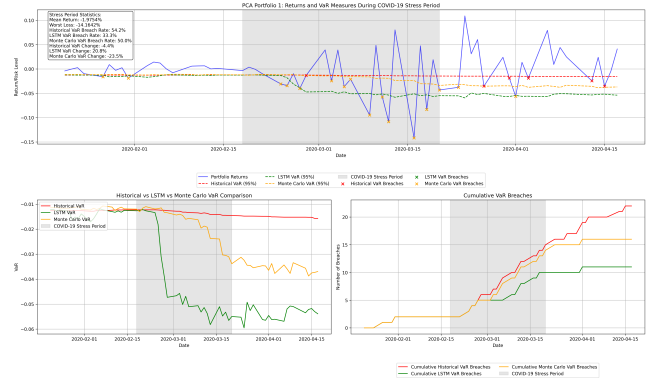


Figure 19: PCA Portfolio 1 risk comparisons (Financial Services, Industrials, Technology heavy) with COVID-19 stress test analysis

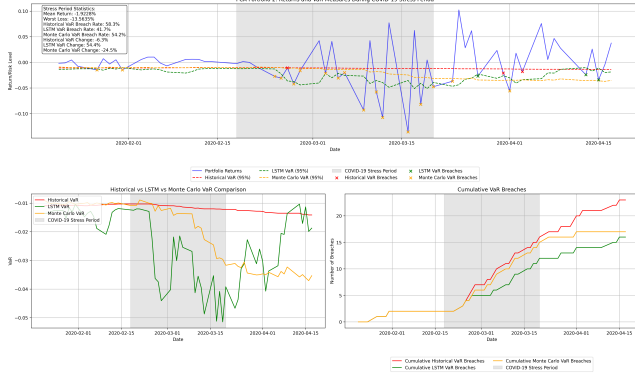


Figure 20: PCA Portfolio 2 risk comparisons (Utilities Heavy) with COVID-19 stress test analysis

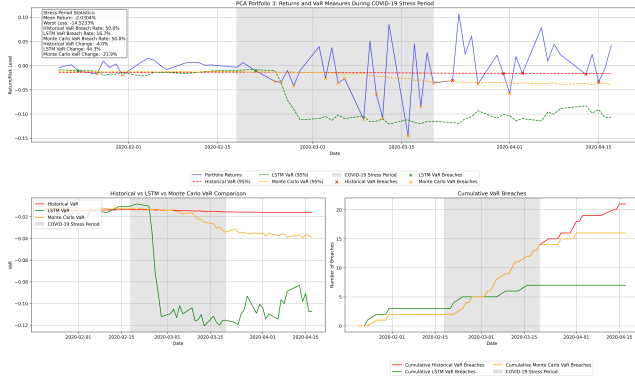


Figure 21: PCA Portfolio 3 risk comparisons (Technology, Financial Services Heavy) with COVID-19 stress test analysis

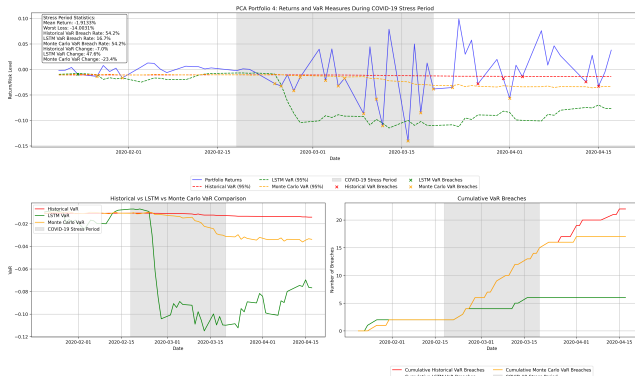


Figure 22: PCA Portfolio 4 risk comparisons (Real Estate Heavy) with COVID-19 stress test analysis

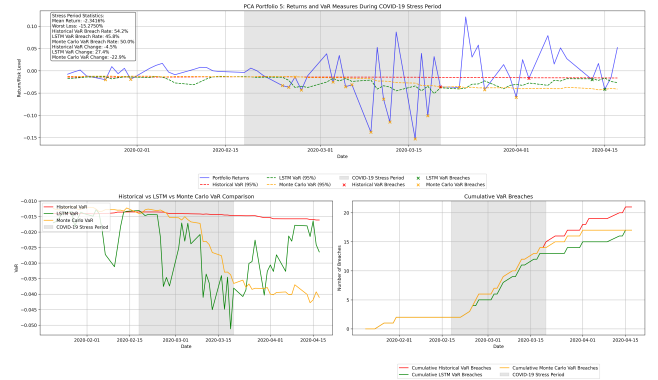


Figure 23: PCA Portfolio 5 risk comparisons (Financial Services, Energy Heavy) with COVID-19 stress test analysis

We can see that evidently, LSTM-VaR is far more responsive to periods of market stress followed by Monte-Carlo VaR. Again we see the strength in the LSTM model in predicting periods of market stress more accurately and with a greater degree of responsiveness than the contemporary methods. VaR breaches for LSTM-VaR however seem to be low, since the VaR thresholds are increased greatly during such periods of shock, again highlighting the counter-intuitiveness of using it as a backtesting measure.

LSTM-VaR also shows the rebound in the COVID-19 financial shock period only benefited certain portfolio combinations. For example, the LSTM-VaR for the market-cap weighted S&P 500 portfolio seems to show a certain level of volatility before showing a strong rebound (reduction in negativity of VaR), which is very much in line with how the markets actually moved for the index during that period of time.

8 Conclusions and Future Work

8.1 Key Findings and Limitations

The above analysis demonstrates that LSTM-based VaR estimation exhibits superior adaptability during periods of market stress, particularly evident during the COVID-19 crisis period where the model achieved lower breach rates in all scenarios compared to historical VaR. The improved performance stems from the model's ability to capture non-linear market dynamics and rapidly adjust risk estimates in response to changing market conditions. However, there were also substantial limita-

tions in the implementation as mentioned in our distribution analysis, where the LSTM-model displays signs of overfitting, evidenced by:

$$\frac{\sigma_{\text{predicted}}}{\sigma_{\text{actual}}} \ll 1 \quad (60)$$

This substantial underestimation of return variability suggests the model is excessively focused on the mean return values rather than capturing the full spectrum of potentially available market dynamics.

The direct modeling of returns along with our simple use of PCA-based simulated portfolios, while computationally convenient, also ignores crucial market characteristics such as the following:

- Long-term price trends and momentum effects
- Temporal dependencies in price levels
- Trading Regime-switching behavior

8.2 Proposed Methodological Improvements and Future Research Directions

In the Financial Risk Management field, The LSTM-VaR framework explored in this paper seems to show particular promise as a stress testing tool. Rather than using it for standard VaR backtesting, it could serve as a dynamic stress period indicator, or a potential stress scenario generator, for use in portfolio stress testing, which is another key component of Financial Risk Management. LSTM models could be used to design stress scenarios that could align with regulatory requirements, however, the explainability of these models would be another important issue to tackle.

To address these limitations, exploring a hybrid LSTM and time series modeling framework that combines the strengths of multiple approaches could be a possible avenue to explore, such as by combining LSTM with ARIMA and GARCH to better model price trends and market volatilities. Instead of directly simulating returns, the P&Ls can instead be calculated from predicted prices and actual prices.

Other avenues for research could include considering the impact on modeling more complex instruments such as fixed income, derivatives and securitized products in a multi-asset class approach to see how the model would behave on different asset classes.

References

- [1] Mina, J., & Xiao, J. X. (2001). *Return to RiskMetrics: The evolution of a standard*. Retrieved from <https://www.msci.com/documents/10199/dbb975aa-5dc2-4441-aa2d-ae34ab5f0945>
- [2] The Investopedia Team. (n.d.). *Backtesting in value at risk (VAR): Meaning, overview, example*. Investopedia. Retrieved from <https://www.investopedia.com/ask/answers/041715/what-backtesting-value-risk-var.asp>
- [3] Li, B. (2023, December 5). *An explanation for the distribution characteristics of stock returns*. Retrieved from <https://arxiv.org/html/2312.02472v1: :text=Observations>
- [4] Sagheer, A., & Kotb, M. (2019, December 13). *Unsupervised pre-training of a deep LSTM-based stacked autoencoder for multivariate time series forecasting problems*. Nature News. Retrieved from <https://www.nature.com/articles/s41598-019-55320-6>
- [5] Gülmez, B. (2023, October 1). *Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm*. Expert Systems with Applications. Retrieved from <https://www.sciencedirect.com/science/article/pii/S0957417423>
- [6] Guijarro-Ordóñez, J., Pelger, M., & Zanotti, G. (2021). *Deep learning statistical arbitrage*. SSRN Electronic Journal. Retrieved from <https://doi.org/10.2139/ssrn.3862004>
- [7] LSTM-based model loss. (n.d.). *Loss function: The training goal aims to reduce... — Download scientific diagram*. Retrieved from https://www.researchgate.net/figure/LSTM-based-model-loss-Loss-Function-The-training-goal-aims-to-reduce-the-loss-model_fig4352502409
- [8] Hayes, A. (n.d.). *Understanding tail risk and the odds of portfolio losses*. Investopedia. Retrieved from <https://www.investopedia.com/terms/t/tailrisk.asp>
- [9] Softplus. Softplus - PyTorch 2.5 documentation. (n.d.). <https://pytorch.org/docs/stable/generated/torch.nn.Softplus.html>
- [10] Bajaj, A. (2024, September 13). Understanding gradient clipping (and how it can fix exploding gradients problem). neptune.ai. <https://neptune.ai/blog/understanding-gradient-clipping-and-how-it-can-fix-exploding-gradients-problem>
- [11] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization.
- [12] A coronavirus stress test for Global Markets. MSCI. (n.d.). <https://www.msci.com/www/blog-posts/a-coronavirus-stress-test-for/01732622890>